
CSE Web Data Collection

Document for 24ada076 - Nuhan

Your part: Ethical scraping and robots.txt, BeautifulSoup, and reading PDFs

Everything behind your part of the project - the ideas, the reasons, and the code - explained step by step, from the beginning.

DA 2009 - Data Collection Methods II

Assignment 2 (Group Project)

University of Colombo

25ada072 - Pasindu | 25ada073 - Sasini | 25ada141 - Salaama | 24ada076 - Nuhan

Contents

How to use this document	3
The project in simple words	4
How the web works	6
The ideas behind your part	9
Reading Python	14
Your file: <code>src/ethics.py</code>	20
Your file: <code>src/static_scraper.py</code>	27
Your file: <code>src/pdf_extractor.py</code>	30
How your part fits into the whole project	33
Showing your part in the viva	34
Practice questions	36
Word list	43

How to use this document

This document explains your part of the project from the very beginning. It does not assume you have seen any of it before. Each chapter builds on the one before it, so read it in order the first time.

Your files are:

- `src/ethics.py`
- `src/static_scraper.py`
- `src/pdf_extractor.py`

You will see four kinds of coloured box:

Think of it like this. a comparison with something from everyday life, to make a new idea easier to picture.

Remember: the one thing to take away from a section. If you only read the green boxes, you will still know the main points.

Be careful: a trap, or something an examiner might test you on.

Try it yourself. something to type on the computer, and what should happen.

Grey boxes in a typewriter font are **code** - real lines copied from our project files, not made-up examples.

A plan for the week before the viva

Day	What to do
1	Read chapters 1 to 3. They explain the project and the ideas behind your part.
2	Read the chapter on reading Python, with your own files open beside it.
3 and 4	Read the chapters on your files, one at a time. Run each file as the 'Try it yourself' boxes show.
5	Read how your part fits into the whole project, and the ethics chapter.
6	Practise the viva chapter out loud - say your 30-second explanation until it feels natural.
7	Go through the practice questions at the end. Cover the answer and try to say it first.

The project in simple words

What a stock exchange is

A company can split its ownership into many small pieces called **shares**. If you buy one share of a bank, you own a tiny piece of that bank.

A **stock exchange** is the marketplace where people buy and sell those shares. The price of a share goes up when many people want to buy it, and down when many people want to sell it. The **Colombo Stock Exchange** - CSE for short - is Sri Lanka's stock exchange, and its website is www.cse.lk.

Think of it like this. a stock exchange is like a big weekly fair where, instead of vegetables, people trade small pieces of companies. The website is the notice board at the entrance that shows today's prices.

What we were asked to do

Our group was given the CSE website and asked to collect data from it with a computer program, using the methods taught in the course, and to do it **ethically** - politely, legally, and without causing the website any trouble.

What we collect

What	What it means	Example
Share prices	What every company traded at today	Sampath Bank: Rs 141.00
Company details	Facts about one company	Its total value, its highest and lowest price this year
Announcements	Notices companies publish, often as PDF files	'A dividend will be paid', 'A meeting will be held'

After collecting, we tidy the data, check it makes sense, draw charts, and save it as files that other people can open.

Who did what

Member	Part of the project
24ada076 - Nuhan (this is you)	Ethical scraping and robots.txt, BeautifulSoup, and reading PDFs
25ada072 - Pasindu	Getting the data from the site, and Selenium
25ada073 - Sasini	Crawling for documents, and cleaning the data
25ada141 - Salaama	OCR, saving the files, and the charts

The app itself (app.py) was put together by all four of us. Each tab in the app is labelled with the member whose part it shows.

The one surprise that shaped the whole project

The first thing we tried was the ordinary method from the course: download the web page and pick the prices out of it. It found almost nothing - about 24 letters of text, and no tables at all.

The reason is that the CSE website is built with **JavaScript**. The page arrives nearly empty, and a small program inside your browser then fetches the prices and fills them in. A plain download never runs that program, so it never sees the prices.

So we opened the site in a browser, pressed **F12**, and watched the **Network** tab, which lists everything the page asks for. It showed the page fetching its prices from addresses such as `https://www.cse.lk/api/tradeSummary`. We read those addresses directly instead, and one request gives us every company at once.

Remember: the CSE page is built by JavaScript, so we read the data from the same addresses the page itself uses. That one decision explains why the project is built the way it is.

How the web works

Before looking at any code, it helps to understand what actually happens when a computer 'visits' a website. There are only a few ideas, and every part of our project uses them.

Requests and responses

When you open a web page, your computer sends a message to another computer, called a **server**, asking for that page. This message is a **request**. The server sends something back - the page, a file, or some data. That is the **response**.

Think of it like this. writing a letter and getting a reply. You write to an address and ask for something. A little later, a reply arrives in your letterbox. Every single thing a website shows you - each picture, each price - arrives this way, one letter and one reply at a time.

Addresses - URLs

Every page and file on the web has an address, called a **URL**. For example:

```
https://www.cse.lk/api/tradeSummary
```

Part	What it means
https://	How to talk to the server. The 's' means the conversation is locked so nobody can read it on the way.
www.cse.lk	Which server - the CSE website.
/api/tradeSummary	Which thing on that server we want. This part is called the path .

GET and POST - two ways of asking

There are two common kinds of request. A **GET** request simply asks for something: 'please give me this page'. A **POST** request sends some information along with the question, like filling in a form before handing it over.

Most of the CSE data addresses only answer **POST** requests - if you send a GET, they reply with an error. That is their rule, not ours, and we found it by trying both.

Status codes - the result stamped on every reply

Every response comes with a three-digit number that says how it went. These were taught in Week 2:

Code	Meaning	In everyday words
200	OK	Here is what you asked for.
301	Moved Permanently	That has moved - look over there.
403	Forbidden	You are not allowed to have this.
404	Not Found	There is no such page.

Code	Meaning	In everyday words
500	Internal Server Error	Something broke on our side.

Headers - the envelope

Along with every request travel a few lines of extra information called **headers**. One of them, the **User-Agent**, says what program is asking. A browser sends something like 'Chrome on Windows'. Our project sends its own name, the university, the course, and an email address.

Think of it like this. the User-Agent is like writing your name and address on the back of the envelope. The receiver knows who wrote, and can write back if there is a problem.

HTML - what a web page is made of

A web page is a text file written in **HTML**. It uses **tags** - words inside angle brackets - to label each part of the page:

```
<html>
  <head>
    <title>Trade Summary</title>
  </head>
  <body>
    <h1>Today's prices</h1>
    <table>
      <tr> <th>Company</th> <th>Price</th> </tr>
      <tr> <td>Sampath Bank</td> <td>141.00</td> </tr>
    </table>
  </body>
</html>
```

Tags usually come in pairs: <title> opens and </title> closes. Whatever sits between them is the content. Tags sit inside other tags, like boxes inside boxes: the table is inside the body, each row <tr> is inside the table, and each cell <td> is inside a row.

JSON - data in a tidy shape

Websites do not only send pages. They also send plain **data**, and the most common format for that is **JSON**. JSON is made of **names and values**, grouped with curly brackets { }, and lists grouped with square brackets []. This is a real piece of what CSE sends us:

```
{
  "symbol": "SAMP.N00000",
  "name": "SAMPATH BANK PLC",
  "price": 141.0,
  "change": 1.0
}
```

Think of it like this. JSON is like a neatly filled-in form. Every box has a label ('name', 'price') and an answer. A program can read it straight away, with no need to search through the decorations of a web page.

Static and dynamic pages

	Static page	Dynamic page
Where the content is	Already written inside the HTML file	Added by JavaScript after the page arrives
Can a plain download see it?	Yes	No
Example	A list of university departments	CSE's live prices

Think of it like this. a static page is a printed menu - everything is already on it. A dynamic page is an empty chalkboard; a waiter writes today's specials on it after you sit down. If you take a photo the moment you walk in, you photograph an empty board.

Try it yourself. open <https://www.cse.lk/equity/trade-summary> in Chrome or Edge. Right-click and choose **View Page Source** - this shows what the server sent, and you will not find the prices in it. Now right-click a price and choose **Inspect** - this shows the page after JavaScript ran, and the prices are there.

The ideas behind your part

Your part rests on three ideas: behaving well on somebody else's website, reading a web page with BeautifulSoup, and getting text out of PDF documents. This chapter explains each idea before we look at any code.

Behaving well on somebody else's website

A website belongs to somebody. Their server costs money to run, and it has to answer everybody who visits. A person clicking around sends a request every few seconds. A program can send hundreds a second if nobody stops it - and a server flooded like that can slow down or stop for everyone else.

So web scraping comes with manners. The Week 6 lecture put them in a table:

Ethical scraping	Unethical scraping
Scrapes only publicly visible data, without getting past logins, CAPTCHAs or paywalls	Gets past logins, paywalls or CAPTCHAs to reach restricted data
Respects the terms of service and follows robots.txt	Ignores the terms and disobeys robots.txt
Uses rate limiting so the server is not overloaded	Sends requests too fast, slowing the site or taking it down
Follows privacy law and avoids personal data	Collects personal data without permission
Credits the source and uses the data fairly	Uses the data with no credit, or to cause harm

Your file, `src/ethics.py`, is where our project does the left-hand column.

robots.txt, line by line

robots.txt is a plain text file that a website keeps at a fixed place - the very top of the site - so that programs always know where to find it. It was agreed as a convention in 1994. It is **voluntary**: nothing stops a program ignoring it. Following it is a choice, and it is the choice ethical scrapers make.

This is CSE's, exactly as they publish it:

```
User-agent: *
Disallow:
Disallow: /cgi-bin/
Sitemap: https://www.cse.lk/sitemap.xml
```

Line	What it means
User-agent: *	The rules below are for every program. The star means 'everyone'. A site can also write separate rules for one named program.
Disallow: (empty)	An empty Disallow blocks nothing. On its own it means 'you may visit everything'.
Disallow: /cgi-bin/	Keep out of any address that starts with /cgi-bin/.
Sitemap: ...	Where the site lists its own pages for programs to read.

Just as important is what is **not** there. There is no `Crawl-delay:` line, which is how a site asks programs to wait a number of seconds between requests. So CSE does not ask us to wait at all. We wait 1.5 seconds anyway, because being allowed to go fast is not a reason to.

Think of it like this. robots.txt is a notice on a shop door: 'Staff only beyond this point'. There is no lock. You stay out because you respect the people who own the shop.

The /cgi-bin/ puzzle - know this one

When you run `python -m src.ethics`, one line of its output looks wrong:

```
yes https://www.cse.lk/cgi-bin/test (allowed)
```

The site says keep out of /cgi-bin/, so why does our check say yes? The answer is how Python's robots.txt reader works. It reads the rules from top to bottom and uses **the first rule that matches**. CSE's empty Disallow: line comes first, and an empty rule matches every address and allows it - so the reader stops there and never reaches the /cgi-bin/ line.

Some other readers, such as Google's, use the most specific rule instead, and they would say no. The site owner almost certainly means 'keep out of /cgi-bin/'. In practice it makes no difference to us, because **none of the addresses our project uses are in /cgi-bin/** - we never go near it.

Be careful: if an examiner spots this line, do not be thrown by it. Explain the first-match rule and the empty Disallow line, and say that we never request anything in /cgi-bin/. Knowing why the output looks odd shows you understand robots.txt better than if the output were simply 'no'.

How websites defend themselves

The Week 6 lecture listed the ways sites protect themselves from badly behaved programs:

Defence	What it does	How our project relates
User-Agent checks	Blocks requests with no name, or a default program name	We send an honest name, so we are easy to recognise
Rate limiting	Limits how many requests one visitor can send	We wait 1.5 s, far slower than any limit
CAPTCHAs	Asks 'are you a person?'	We never met one, and would not try to get past one
IP blocking	Bans a computer's address	We never gave them a reason to

Remember: we are not trying to sneak past any of these. We behave well enough that none of them ever needed to act.

Reading a web page with BeautifulSoup

requests is the library that sends a request and brings back the reply. If the reply is a web page, what you get is one very long piece of HTML text. To find the parts you want in it, you need something that understands HTML. That is **BeautifulSoup**.

BeautifulSoup reads the HTML and builds a **tree** - a map of which tags sit inside which. Then you can ask it questions like 'find the title' or 'find every table'. Reading the text and building the tree is called **parsing**, and the part of BeautifulSoup that does it is the **parser**.

```
html
+-- head
|   +-- title      'Trade Summary - CSE 2025'
+-- body
    +-- h1         'Today's prices'
    +-- table
        +-- tr
        |   +-- th 'Company'   +-- th 'Price'
        +-- tr
            +-- td 'Sampath'   +-- td '141.00'
```

Think of it like this. the HTML is a long letter with no paragraphs. BeautifulSoup turns it into a family tree, so you can say 'find me the table, and every row inside it' instead of reading the whole letter yourself.

The methods from the Week 4 lecture:

Method	What it does	Example
<code>find()</code>	The first tag that matches	<code>soup.find('title')</code>
<code>find_all()</code>	Every tag that matches, as a list	<code>soup.find_all('a')</code>
<code>get_text()</code>	The words inside a tag	<code>tag.get_text()</code>
<code>get()</code>	The value of one attribute	<code>tag.get('href')</code>
<code>select()</code>	Every match for a CSS selector	<code>soup.select('div > p')</code>
<code>select_one()</code>	The first CSS selector match	<code>soup.select_one('.title')</code>

An **attribute** is extra information inside a tag. In `<a href="https://www.cse.lk">`, the attribute is `href` and its value is the address the link goes to.

Getting text out of PDFs

A **PDF** is a document format designed to look exactly the same on every computer and printer. It keeps the fonts, the layout and the pictures precisely where they were put.

The surprising thing is that a PDF is not really a text document at all. Inside, it is a list of drawing instructions: 'put the letter D at this spot, the letter E just to its right, draw a line here'. There is no instruction that says 'this is a paragraph' or 'this is a table'. A PDF library has to work those out by looking at where the letters sit.

The three kinds of PDF

Kind	What is inside	Can we pull the text out?
Text-based	Real letters, made by a computer (for example saved from Word)	Yes
Image-based (scanned)	A photograph of a printed page	No - there are no letters inside, only a picture. It needs OCR.
Hybrid	Some pages of each kind	Only from the typed pages

We have real examples of all three among the CSE documents we downloaded:

Document	Pages	Characters found	Kind
Extension of the final suspension date	1	3,267	Text-based
Employee share option schemes	1	896	Text-based
De-listing - Associated Motor Finance	1	0	Scanned
Revised format for disclosure ...	4	0, 127, 1,129, 274	Hybrid - page 1 is a scan



The de-listing notice. It looks like an ordinary document, but it is a photograph of paper - there is not a single letter of text inside the file.

Think of it like this. a text-based PDF is a typed letter - you can copy any word out of it. A scanned PDF is a photo of that letter - you can see the words, but to the computer it is just coloured dots.

Why getting text out is harder than it sounds

The Week 7 lecture listed the problems:

- **Tables** can run over several pages and have merged cells.
- **Paragraphs** - a line break might mean a new paragraph, or just the edge of the page.
- **Headers and footers** repeat on every page, and you may not want them.
- **Formatting** such as bold, italics and joined letters can come out wrong.
- **Scanned pages** have no text at all.

PyPDF2 and pdfplumber

	PyPDF2	pdfplumber
Good at	Simple text, and the file's details (pages, author)	Text that keeps its layout, and tables
Layout	Often merges lines and loses structure	Uses the position of each letter to keep structure
Tables	Cannot find them	Can find and extract them
We use it for	The file details	The main text, and tables

Remember: we use both because they are good at different things: pdfplumber for the text and tables, PyPDF2 for the file details. And no library at all can read a scanned page - that is OCR's job.

Reading Python

To explain your code, you first need to be able to read it. This chapter explains every piece of Python that appears in your files. Every example is a real line from your own code, so by the end of the chapter you will recognise all of it.

Try it yourself. open `src/ethics.py` in VS Code or Notepad and keep it beside this document while you read. Find each example in the file as you go.

Comments - notes for people

A line that starts with `#` is a **comment**. Python ignores it completely. It is there for people reading the code. Our files are full of comments explaining why each part exists.

From your code:

```
# If we cannot read the rules we do not guess - we stop.
```

From `can_fetch()`. Python skips this line - it explains to a reader why the next line returns `False`.

Variables - labelled boxes

A **variable** is a name that holds a value. The `=` sign means 'put the thing on the right into the box on the left'. It does not mean 'equals' the way it does in maths.

Think of it like this. a variable is a labelled box. `price = 141.0` means: take a box, write 'price' on the label, and put 141.0 inside.

From your code:

```
since_last = time.time() - self.last_request_time
```

From `wait()`. `time.time()` is the current time in seconds. Take away the time of the last request, and the answer - how many seconds have passed - goes into a box called `since_last`.

Text, and filling in blanks with f-strings

Text in Python is written inside quote marks, like `"hello"`. It is called a **string**. A string that starts with `f` is an **f-string**: anything inside curly brackets `{ }` is replaced with the value of that variable.

Think of it like this. an f-string is a form letter with blanks. `f"Dear {name}"` fills in the blank with whatever name holds.

From your code:

```
self.log.add(f"SKIPPED {url} - {reason}")
```

From `fetch()`. If `url` holds `https://www.cse.lk/cgi-bin/x` and `reason` holds `not allowed by robots.txt`, the message becomes `SKIPPED https://www.cse.lk/cgi-bin/x - not allowed by robots.txt`.

Numbers, and True and False

Python has whole numbers like `30`, numbers with a decimal point like `1.5`, and two special values: `True` and `False`. `None` means 'nothing here yet'.

From your code:

```
return True, "allowed"
```

From `can_fetch()`. The function hands back two things at once: `True` (yes, we may) and a short reason. The code that asked catches both: `allowed, reason = self.can_fetch(url)`.

Lists - things in order

A **list** holds several values in order, inside square brackets. `.append()` adds one more to the end. `[]` on its own is an empty list, ready to be filled.

Think of it like this. a list is a numbered shopping list. You can add items to the end, and read them back in order.

From your code:

```
paths = []
for line in self.robots_text.splitlines():
    ...
    paths.append(value)
```

From `blocked_paths()`. Start with an empty list, then add each blocked folder found in `robots.txt`. For CSE the finished list is `['/cgi-bin/']`.

Dictionaries - values with labels

A **dictionary** holds values under names, inside curly brackets: `{"name": value}`. You get a value back by its name: `reply["ok"]`. The method `.get(name, default)` does the same, but gives back the default instead of crashing if the name is missing.

Think of it like this. a dictionary is a set of labelled drawers. You open a drawer by reading its label, not by counting from the top.

From your code:

```
return {"ok": True, "data": result, "from_file": False,
        "error": None}
```

From `fetch()`. Every request hands back a dictionary with the same four labels, so the code that asked can check `reply["ok"]` first and then read `reply["data"]`.

Functions - recipes

A **function** is a named set of steps. `def` starts one. The names in brackets are its **parameters** - the ingredients it needs. `return` hands back the result. The function does nothing until somebody **calls** it by writing its name followed by brackets.

Think of it like this. a function is a recipe. The parameters are the ingredients, the indented lines are the steps, and `return` is the finished dish.

From your code:

```
def page_type(text):
    length = len(text.strip())
    if length < config.OCR_TEXT_THRESHOLD:
        return "Scanned image", f"Only {length} characters - needs OCR"
    return "Text based", f"{length}, characters read from the page"
```

From `pdf_extractor.py`. The ingredient is the text from one page. `len()` counts its characters, and the function returns what kind of page it is.

if and else - making decisions

`if` runs the indented lines only when a condition is true. `else` runs when it is not. `not` turns true into false and the other way round.

From your code:

```
if since_last < self.delay:
    time.sleep(self.delay - since_last)
```

From `wait()`. Only if less than 1.5 seconds have passed do we sleep - and only for the time that is left.

for loops - doing the same thing for each item

A **for loop** repeats its indented lines once for every item in a list or table. `enumerate(..., start=1)` also counts them for you: 1, 2, 3.

Think of it like this. a for loop is a teacher going down the register, doing the same thing for each name.

From your code:

```
for number, page in enumerate(reader.pages, start=1):
    text = page.extract_text() or ""
```

From `read_with_pypdf2()`. Go through the pages one at a time. `enumerate` numbers them 1, 2, 3 so we can report 'page 2'.

import - taking a toolbox off the shelf

Python comes with some tools built in. For everything else we **import** a library - a toolbox somebody else wrote. `import pandas as pd` means 'load pandas, and let me call it `pd` for short'. `from X import Y` takes just one tool out of the box.

From your code:

```
import requests
from urllib.robotparser import RobotFileParser
from bs4 import BeautifulSoup
import pdfplumber
from PyPDF2 import PdfReader
```

Your three files use these toolboxes: requests to send requests, RobotFileParser to understand robots.txt, BeautifulSoup to read HTML, and pdfplumber and PyPDF2 to read PDFs.

try and except - a safety net

Code inside try is attempted. If anything goes wrong, Python jumps to except instead of crashing. We use this wherever the internet or a file might fail.

Think of it like this. try and except is a trapeze artist with a safety net. They attempt the jump; if they fall, the net catches them and the show goes on.

From your code:

```
try:
    response = requests.get(config.ROBOTS_URL, ...)
    ...
except Exception as e:
    self.robots = None
```

From read_robots(). If the internet is down and robots.txt cannot be read, the program does not crash - it notes that it has no rules, and from then on refuses every request.

with - borrow, use, and give back automatically

with opens something - usually a file - for the indented lines that follow, and closes it again afterwards, even if something goes wrong in the middle.

Think of it like this. borrowing a library book with a promise that it is returned automatically, even if you forget.

From your code:

```
with pdfplumber.open(str(pdf_path)) as pdf:
    for number, page in enumerate(pdf.pages, start=1):
        text = page.extract_text() or ""
```

From read_with_pdfplumber(). The PDF is opened for the indented lines only, and closed automatically at the end - so no file is left open on the computer.

Classes, objects and self

A **class** is a design for making things that hold both information and actions. The thing you make from the design is an **object**. Functions written inside a class are called **methods**. `__init__` is the special method that runs once, when the object is first made - it sets it up. Inside a class, **self** means 'this particular object'.

Think of it like this. a class is a cake tin, and each object is a cake made from it. `self` means 'this cake' - so `self.delay` is this cake's own delay, not somebody else's.

From your code:

```
class Scraper:

    def __init__(self, delay=config.DELAY):
        self.delay = delay
        self.log = ScraperLog()
        self.last_request_time = 0
```

From `ethics.py`. `Scraper` is the design. When the app writes `Scraper()`, one scraper object is made, and `__init__` gives it its own delay, its own log, and a note of when it last sent a request. Your file has two classes: `ScraperLog` and `Scraper`.

pandas DataFrames - tables in Python

A **DataFrame** is a table with rows and named columns, like a sheet in Excel. `pandas` is the library that provides it. `pd.DataFrame(rows)` builds one from a list of dictionaries, `df.head(5)` shows the first five rows, and `df["price"]` picks out one column.

From your code:

```
return pd.DataFrame(rows)
```

From `scrape_sitemap()` and `check_all_pdfs()`. `rows` is a list of dictionaries, one per sitemap page or per PDF, and this line turns it into a table the app can display.

Two tricks worth knowing

or **as a fallback**. `x or y` gives `x`, unless `x` is empty or `None`, in which case it gives `y`. We use it where a library might hand back `None`:

```
text = page.extract_text() or ""
```

If `extract_text()` returns `None` - which it does for an empty page - `text` becomes an empty string instead, so later lines such as `len(text)` do not crash.

@staticmethod

A line starting with `@` just above a function changes how it behaves. `@staticmethod` means the method does not need `self` - it does not use anything belonging to one particular scraper:

```
@staticmethod
def save_copy(name, data):
    path = config.CACHE_DIR / f"{name}.json"
```

Saving a copy of a reply works the same for every scraper, so it does not need to know which one called it.

Running a file

Every file in our project can be run on its own to show what it does. You open a terminal in the project folder, switch on the project's Python, and ask Python to run the file:

```
cd D:\DA2009\CSE-Scraping-Project
.venv\Scripts\activate
python -m src.ethics
```

.venv is the project's own copy of Python with all the libraries installed. -m src.ethics means 'run the file ethics.py inside the src folder'. The part at the bottom of each file that starts if __name__ == "__main__": is what runs when you do this.

Your file: src/ethics.py

Every request our project sends to the internet goes through this file. It is where the ethical rules live.

Putting the rules in one place is the most important design decision in the project. If each file sent its own requests, each one would have to remember to check robots.txt, to wait, and to log. Forget it once, in one file, and the project breaks its own rules. With one door, there is nothing to forget.

Think of it like this. a building where every visitor must go through the one front desk. The desk checks the visitor list, makes you wait your turn, signs you in, and writes you in the book. There is no side door to forget to lock.

The toolboxes at the top

```
import csv
import json
import time
from datetime import datetime
from urllib.robotparser import RobotFileParser

import requests

import config
```

Import	Used for
csv	Writing the request log as a CSV file
json	Saving and loading copies of the replies
time	Measuring time, and sleeping for the delay
datetime	Writing the date and time on each log line
RobotFileParser	Understanding robots.txt - it comes with Python
requests	Actually sending the requests
config	Our own settings file: the delay, the User-Agent, the addresses

Part 1 - ScraperLog, the record book

This class keeps two records: the messages shown in the app, and a permanent CSV file with one line per request.

```
class ScraperLog:

    def __init__(self):
        self.messages = []
        self.request_count = 0

    def add(self, message):
        stamp = datetime.now().strftime("%H:%M:%S")
        self.messages.append(f"{stamp} {message}")
```

```
return self.text()
```

`add()` puts the time in front of every message, so the app shows lines like `17:47:55 Read robots.txt` from the website. `strftime("%H:%M:%S")` turns the time into hours:minutes:seconds.

```
def log_request(self, method, url, status, allowed):
    self.request_count += 1
    new_file = not config.REQUEST_LOG.exists()

    with open(config.REQUEST_LOG, "a", newline="",
              encoding="utf-8") as f:
        writer = csv.writer(f)
        if new_file:
            writer.writerow(["time", "method", "url",
                            "status", "robots_allowed",
                            "delay_seconds"])
        writer.writerow([ ...the values... ])
```

Line by line:

- `self.request_count += 1` - add one to the count. `+=` means 'add to what is already there'.
- `new_file = not config.REQUEST_LOG.exists()` - is this the first time? If the file does not exist yet, we will need a heading row.
- `open(..., "a")` - the "a" means **append**: add to the end of the file instead of wiping it. That is why the log keeps growing across every run of the project.
- The heading row is written only once, then one row per request.

This is what the log looks like - these are real rows:

```
time,method,url,status,robots_allowed,delay_seconds
2026-09-26 17:47:17,GET,https://www.cse.lk/sitemap.xml,200,True,1.5
2026-09-26 17:47:31,GET,https://www.cse.lk/equity/trade-summary,200,True,1.5
```

Remember: the request log is our evidence. If anybody asks whether we overloaded CSE's server, we open this file and show them every request, when it was sent, and that we waited 1.5 seconds before each one.

Part 2 - Setting up a Scraper

```
class Scraper:
```

```
    def __init__(self, delay=config.DELAY):
        self.delay = delay
        self.log = ScraperLog()
        self.last_request_time = 0

        self.session = requests.Session()
        self.session.headers.update(
            {"User-Agent": config.USER_AGENT})

        self.robots_text = ""
        self.robots = None
        self.read_robots()
```

- `delay=config.DELAY` - the delay is 1.5 seconds unless somebody asks for a different one.

- `self.last_request_time = 0` - no request has been sent yet.
- `requests.Session()` - a **session** keeps the connection to the server open between requests, instead of opening a fresh one every time. That is a little less work for CSE's server.
- `headers.update({"User-Agent": ...})` - write our name on every request this session sends, once, here.
- `self.read_robots()` - read robots.txt straight away, before any other request can be made.

The User-Agent it sends, from `config.py`:

```
CSE-Academic-Scraper/1.0 (University of Colombo; DA 2009 group
project; contact: nuhanmalee@gmail.com)
```

Many scraping tutorials tell you to copy a browser's User-Agent so the website cannot tell you apart from a person. We do the opposite. The point of the header is to say who is asking, not to hide it.

Part 3 - Reading robots.txt

```
def read_robots(self):
    try:
        response = requests.get(
            config.ROBOTS_URL,
            headers={"User-Agent": config.USER_AGENT},
            timeout=config.TIMEOUT,
        )
        self.robots_text = response.text

        self.robots = RobotFileParser()
        self.robots.parse(self.robots_text.splitlines())

        self.log.add("Read robots.txt from the website")

    except Exception as e:
        self.robots = None
        self.log.add(f"Could not read robots.txt: {e}")
```

- `requests.get(config.ROBOTS_URL, ...)` - download <https://www.cse.lk/robots.txt>. `timeout=30` means give up after 30 seconds rather than waiting forever.
- `response.text` - the reply as text: the four lines we saw earlier.
- `RobotFileParser()` then `.parse(...)` - hand those lines to Python's own robots.txt reader. We did not write the rule-matching ourselves; the standard library already knows the format.
- `.splitlines()` - split the text into a list of separate lines, which is what the parser wants.

This is the one request in the project that does **not** go through `fetch()` - and it cannot, because `fetch()` checks robots.txt first, and we cannot check robots.txt before we have read it.

The except part is the careful bit. If robots.txt cannot be read, we set `self.robots = None`. From then on every permission check answers no. When we cannot see the rules, we do not guess in our own favour.

Part 4 - Asking permission

```
def can_fetch(self, url):
    if self.robots is None:
        # If we cannot read the rules we do not guess - we stop.
        return False, "robots.txt could not be read"
```

```

if self.robots.can_fetch(config.USER_AGENT, url):
    return True, "allowed"
return False, "not allowed by robots.txt"

```

Give it an address, and it answers yes or no, with a reason. `self.robots.can_fetch(...)` is the standard library doing the real work: it compares our User-Agent and the address with the rules it read.

`blocked_paths()` just below it reads the Disallow lines so the app can show them:

```

def blocked_paths(self):
    paths = []
    for line in self.robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            value = line.split(":", 1)[1].strip()
            if value:
                paths.append(value)
    return paths

```

- `.strip()` removes spaces at both ends of the line.
- `.lower().startswith("disallow:")` - does this line start with 'disallow:', ignoring capital letters?
- `.split(":", 1)[1]` - cut the line at the first colon and take the part after it. Disallow: /cgi-bin/ becomes /cgi-bin/.
- `if value:` - skip the empty Disallow: line, which blocks nothing.

Part 5 - Waiting politely

```

def wait(self):
    since_last = time.time() - self.last_request_time
    if since_last < self.delay:
        time.sleep(self.delay - since_last)
    self.last_request_time = time.time()

```

It does not blindly sleep 1.5 seconds every time. It measures how long it has been since the last request, and sleeps only for what is left. A worked example:

Time since last request	Is it less than 1.5?	Sleeps for
0.2 seconds	Yes	1.3 seconds
1.0 seconds	Yes	0.5 seconds
4.0 seconds	No	0 seconds - go straight away

Either way the server gets at least 1.5 seconds between our requests, and we never waste time waiting longer than we promised.

Think of it like this. not ringing a doorbell again and again. You ring, you wait a moment, and if you have already been waiting on the step for a while, you do not need to wait any longer.

`set_delay()` lets the delay be changed, but only within safe limits:

```

def set_delay(self, seconds):
    self.delay = max(config.MIN_DELAY,

```

```
min(float(seconds), config.MAX_DELAY))
```

`min(x, 10)` means 'no more than 10', and `max(0.5, ...)` means 'no less than 0.5'. Ask for 0.1 and you get 0.5. Ask for 50 and you get 10. Nobody can switch the politeness off.

Part 6 - Keeping a copy of every reply

```
@staticmethod
def save_copy(name, data):
    path = config.CACHE_DIR / f"{name}.json"
    with open(path, "w", encoding="utf-8") as f:
        json.dump(data, f)
```

Each reply is saved as a file in `data/cache/`, named after the address it came from - for example `data/cache/tradeSummary.json`. `load_copy()` reads it back. The `/` between the folder and the file name joins them into one path.

Why keep copies? The Week 6 lecture says 'avoid over-requesting - don't repeatedly fetch unchanged content'. And if the internet fails during a demonstration, the app can show the saved copy instead of an error.

Part 7 - fetch(), the one door

Everything above leads to this function. Read it in four steps:

```
def fetch(self, url, method="GET", data=None, save_as=None,
          want="json"):

    #--- Step 1: are we allowed? ---
    allowed, reason = self.can_fetch(url)
    if not allowed:
        self.log.add(f"SKIPPED {url} - {reason}")
        self.log.log_request(method, url, "skipped", allowed)
        return {"ok": False, "data": None, "from_file": False,
                "error": "robots.txt says we should not ..."}

    #--- Step 2: wait ---
    self.wait()

    #--- Step 3: send it ---
    try:
        if method == "POST":
            response = self.session.post(url, data=data,
                                         timeout=config.TIMEOUT)
        else:
            response = self.session.get(url, timeout=config.TIMEOUT)

    #--- Step 4: record it ---
    self.log.log_request(method, url, response.status_code,
                        allowed)
```

Step	What happens	Why
1. Permission	Ask robots.txt. If the answer is no, stop and record that we skipped it.	It is the first line - no request can get past it
2. Wait	Sleep until 1.5 seconds have passed	Politeness

Step	What happens	Why
3. Send	GET or POST, through the session with our User-Agent	Some CSE addresses only answer POST
4. Record	Write a row in the request log	Evidence

Then it looks at the reply:

```

if response.status_code != 200:
    return {"ok": False, ...,
            "error": f"The server replied with "
                    f"{response.status_code}."}

if want == "bytes":
    return {"ok": True, "data": response.content, ...}

if want == "text":
    return {"ok": True, "data": response.text, ...}

result = response.json()
if save_as:
    self.save_copy(save_as, result)
return {"ok": True, "data": result, ...}

```

!= means 'is not equal to'. Anything other than 200 is treated as a failure and reported, not retried again and again. Then want decides what shape the reply is handed back in:

want	Gives back	Used by
"json" (the default)	Python lists and dictionaries, made from the JSON reply	api_client.py - the market data
"text"	The reply as a string	static_scraper.py - web pages and the sitemap
"bytes"	The raw contents of a file	crawler.py - PDF downloads

And if anything goes wrong at all:

```

except Exception as e:
    self.log.add(f"Request failed: {e}")
    if save_as:
        saved = self.load_copy(save_as)
        if saved is not None:
            return {"ok": True, "data": saved,
                    "from_file": True, "error": None}
    return {"ok": False, "data": None, "from_file": False,
            "error": str(e)}

```

If the internet is down, use the saved copy if there is one, and say so with from_file: True. Otherwise report the error. The program never crashes - which matters because it runs behind a web page, and a crash would close the whole app.

Remember: fetch() does four things in a fixed order - check robots.txt, wait, send, record - and the check comes first. Because it is the only function that sends requests, no request can skip the rules.

Be careful: there is one honest exception. `src/selenium_scraper.py` drives a real web browser, and the browser makes its own requests that never pass through `fetch()`. We give it the same User-Agent and use it on one page only, but if asked 'does every request go through `fetch()`?', the true answer is 'every request except Selenium's browser'. Saying so shows you understand the design.

Part 8 - The rules shown in the app

At the bottom of the file is `ETHICAL_RULES`, a list of pairs: a rule and what we do about it. The Ethics tab shows it as a table. Each pair is written `( "name", "explanation" )` - round brackets hold a fixed pair of values, called a **tuple**.

```
ETHICAL_RULES = [  
    ("Check robots.txt",  
     "We read the robots.txt file and check every address ..."),  
    ("Wait between requests", ...),  
    ("Say who we are", ...),  
    ...  
]
```

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.ethics  
  
robots.txt from CSE  
-----  
User-agent: *  
Disallow:  
Disallow: /cgi-bin/  
Sitemap: https://www.cse.lk/sitemap.xml  
  
Paths the website asks us not to visit: ['/cgi-bin/']  
  
Permission checks  
-----  
yes https://www.cse.lk/api/aspiData (allowed)  
yes https://www.cse.lk/equity/trade-summary (allowed)  
yes https://www.cse.lk/cgi-bin/test (allowed)  
  
A real request  
-----  
ok : True  
data: {'id': 37116752, 'value': 21037.35, ...}
```

The third permission line is the `/cgi-bin/` puzzle from the ideas chapter. The last part is a real request for the market index (ASPI), sent through `fetch()`.

Your file: src/static_scraper.py

The ordinary way of scraping - download the page, read it with BeautifulSoup - and why it finds almost nothing on the CSE site.

This file does not collect the project's data. It is here because of what it shows. Without it, a reader would reasonably ask why we did not just use BeautifulSoup like everybody else. This file answers that in one run.

Part 1 - scrape_page()

```
def scrape_page(url=None, scraper=None):
    scraper = scraper or Scraper()
    url = url or config.PAGES["trade_summary"]

    reply = scraper.fetch(url, want="text")

    if not reply["ok"]:
        return {"url": url, "status": None,
                "error": reply["error"], ...}

    html = reply["data"]
    status = 200
```

- `scraper or Scraper()` - use the scraper we were given, or make a new one if none was given.
- `url or config.PAGES["trade_summary"]` - if no address was given, use the CSE trade summary page.
- `scraper.fetch(url, want="text")` - the request goes through your own `fetch()`, so `robots.txt` is checked, the delay applied and the request logged. `want="text"` gives us the HTML as one string.
- `status = 200` - `fetch()` only says ok when the server replied 200, so we know the status.

Then BeautifulSoup reads it:

```
soup = BeautifulSoup(html, "html.parser")

title = soup.title.get_text(strip=True) if soup.title \
    else "no title"
words = soup.get_text(strip=True)
tables = soup.find_all("table")
links = soup.find_all("a")
```

- `BeautifulSoup(html, "html.parser")` - build the tree. `html.parser` is the parser that comes with Python, so nothing extra is needed.
- `soup.title.get_text(strip=True)` - the words inside the `<title>` tag, with spaces trimmed.
- `soup.get_text(strip=True)` - every word on the whole page.
- `soup.find_all("table")` - every table. `len()` of that list is how many there are.

The result on the CSE trade summary page:

```
Address      : https://www.cse.lk/equity/trade-summary
Status       : 200
HTML size    : 17,898 bytes
Words found: 24 characters
Tables       : 0
```

The request worked perfectly - status 200 - and nearly 18,000 bytes of HTML came back. (The CSE home page gives about 25,000.) Yet BeautifulSoup finds only 24 characters of words and not one table. The prices are simply not in the file.

Think of it like this. you asked for a newspaper and got the paper with all the pages blank - the printing happens later, inside the reader's browser. requests brings you the blank paper. BeautifulSoup reads it perfectly well; there is just nothing printed on it yet.

Part 2 - scrape_sitemap(), the proof

The obvious question is: are you sure BeautifulSoup was not the problem? This function answers it. It uses the same library on CSE's sitemap, which is an ordinary file with no JavaScript.

```
def scrape_sitemap(scraper=None):
    scraper = scraper or Scraper()

    reply = scraper.fetch(f"{config.BASE_URL}/sitemap.xml",
                        want="text")
    if not reply["ok"]:
        return pd.DataFrame()

    try:
        soup = BeautifulSoup(reply["data"], "xml")
    except Exception:
        soup = BeautifulSoup(reply["data"], "html.parser")

    rows = []
    for entry in soup.find_all("url"):
        location = entry.find("loc")
        updated = entry.find("changefreq")
        rows.append({
            "Page": location.get_text(strip=True) if location else None,
            "Updated": updated.get_text(strip=True) if updated else None,
        })
    return pd.DataFrame(rows)
```

A sitemap is written in **XML**, which looks like HTML but with tags the website makes up itself. Each page in the list looks like this:

```
<url>
  <loc>https://www.cse.lk/market-summary/</loc>
  <changefreq>daily</changefreq>
</url>
```

- BeautifulSoup(..., "xml") - read it as XML. That needs the lxml library, so if lxml is missing, except falls back to the normal parser instead of crashing.
- soup.find_all("url") - every <url> block.
- entry.find("loc") - inside each one, the <loc> tag holding the address.
- ... if location else None - if a block has no <loc>, write None instead of crashing.

Result: **15 pages read without any trouble.** Same library, same methods, a perfect result - so the earlier problem was the page, not the tool.

Part 3 - beautifulsoup_methods()

This last function runs each Week 4 method on the real CSE page and shows what it gave back. It is useful for revising, and for answering 'show me you know BeautifulSoup':

Method	Code we ran	What it gave back
find()	soup.find('title')	<title>Trade Summary - CSE 2025</title>
find_all()	len(soup.find_all('a'))	0 - no links at all
get_text()	soup.get_text(strip=True)[:40]	Trade Summary - CSE 2025
get()	soup.find('a').get('href')	no links to read
select()	len(soup.select('meta'))	3
select_one()	soup.select_one('meta[name=viewport]')	the viewport meta tag

Notice that nearly all the 24 characters of text are just the page title. That is everything a plain download can see.

Try it yourself. run it yourself:

```
python -m src.static_scraper
```

Remember: requests downloaded the page correctly and BeautifulSoup read it correctly. The page itself had almost nothing in it, because CSE builds its content with JavaScript afterwards. That finding is why the rest of the project reads the data addresses instead.

Your file: src/pdf_extractor.py

Opens the announcement PDFs, pulls the text out, and works out which ones are scans that need OCR instead.

Part 1 - Is this page real text, or a scan?

```
def page_type(text):
    length = len(text.strip())

    if length < config.OCR_TEXT_THRESHOLD:
        return "Scanned image", f"Only {length} characters - needs OCR"
    return "Text based", f"{length:,} characters read from the page"
```

Count the characters on a page. Fewer than 50, and we call it a scan. {length:,} writes the number with a comma, so 3267 appears as 3,267.

Why 50 and not 0? A real page of a CSE circular gives hundreds or thousands of characters, and a scan gives none - so almost any number would work. But a scan sometimes carries a few stray characters that are not the real text - a label stamped on afterwards, for instance. Demanding at least 50 means a few stray letters cannot fool us into skipping OCR.

Part 2 - Reading with PyPDF2

```
def read_with_pypdf2(pdf_path):
    reader = PdfReader(str(pdf_path))

    pages = []
    for number, page in enumerate(reader.pages, start=1):
        text = page.extract_text() or ""
        kind, note = page_type(text)
        pages.append({"Page": number,
                     "Characters": len(text.strip()),
                     "Type": kind, "Note": note, "text": text})

    details = reader.metadata or {}
    info = {
        "Pages": len(reader.pages),
        "Title": details.get("/Title", "not set"),
        "Author": details.get("/Author", "not set"),
        "Created with": details.get("/Producer", "not set"),
    }
    return pages, info
```

- PdfReader(...) - open the PDF. reader.pages is the list of its pages.
- page.extract_text() or "" - the text on this page. For an empty page PyPDF2 gives back None, and the or "" turns that into an empty string so nothing below crashes.
- reader.metadata - the **metadata**: facts stored inside the file about the file, such as who wrote it and what program made it.
- details.get("/Author", "not set") - the author, or 'not set' if the file does not say.

Real metadata from one CSE circular:

Pages	1
Title	not set

'Created with Microsoft Word' is a second clue that this is a typed, text-based PDF and not a scan.

Part 3 - Reading with pdfplumber

```
def read_with_pdfplumber(pdf_path):
    pages = []
    tables = []

    with pdfplumber.open(str(pdf_path)) as pdf:
        for number, page in enumerate(pdf.pages, start=1):
            text = page.extract_text() or ""
            kind, note = page_type(text)
            pages.append({...})

            for table in (page.extract_tables() or []):
                if len(table) > 1:
                    heading = [(cell if cell else f"Column {i + 1}")
                               for i, cell in enumerate(table[0])]
                    tables.append(pd.DataFrame(table[1:],
                                              columns=heading))

    return pages, tables
```

- `with pdfplumber.open(...) as pdf:` - open the PDF, and close it automatically afterwards.
- `page.extract_tables()` - pdfplumber looks at where the text sits and finds tables. Each table comes back as a list of rows, and each row as a list of cells.
- `if len(table) > 1:` - a table needs a heading row and at least one row of data.
- `table[0]` is the heading row, and `table[1:]` is every row after it. `[1:]` means 'from position 1 to the end'.
- An empty heading cell becomes 'Column 1', 'Column 2' and so on, so every column has a name.

The line that builds heading is a **list comprehension** - a short way to build a list in one line. It reads: 'for each cell in the first row, use the cell, or Column n if it is empty'.

Part 4 - read_pdf() puts it together

```
def read_pdf(pdf_path):
    ...
    try:
        pages, tables = read_with_pdfplumber(pdf_path)
        _, info = read_with_pypdf2(pdf_path)
    except Exception as e:
        return {"ok": False, "file": name, "error": str(e), ...}

    scanned = [p["Page"] for p in pages
               if p["Type"] == "Scanned image"]
```

- pdfplumber gives the pages and tables. PyPDF2 gives the file details. `_, info = ...` means 'I only want the second thing it returns' - the `_` is a name for something we ignore.
- `scanned` is the list of page numbers that need OCR. For the de-listing notice it is `[1]`. For the hybrid document it is also `[1]` - its first page is a scan, and the other three are typed.

The OCR tab uses exactly that list, so for a hybrid PDF it reads the page that is actually scanned, not just page 1.

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.pdf_extractor
```

File	Pages	Characters	Type	Needs OCR
14336_EXTENSION OF THE FINAL SUSPENSION	1	3267	Text based	No
14358_EMPLOYEE SHARE OPTION SCHEMES	1	896	Text based	No
14364_DE-LISTING OF THE ORDINARY VOTING	1	0	Scanned image	Yes
14396_THE REVISED FORMAT FOR DISCLOSURE	4	1530	Scanned image	Yes

Look at the row with 0 characters: that file is about 190 KB and full of writing when you open it, but there is no text inside it. The 14396 row shows 1,530 characters and still needs OCR - that is the hybrid one, with a scanned first page.

Remember: PyPDF2 and pdfplumber can only read letters that are really in the file. A scanned page is a picture, so they get nothing - and that is when OCR takes over.

How your part fits into the whole project

Every button in the app follows the same path. The lines marked <-- YOU are yours.

1. You click a button in the app	app.py	
2. The app asks for some data	src/api_client.py	
3. Every request goes through fetch()	src/ethics.py	<-- YOU
a. check robots.txt	src/ethics.py	<-- YOU
b. wait 1.5 seconds	src/ethics.py	<-- YOU
c. send it, saying who we are	src/ethics.py	<-- YOU
d. write it in the request log	src/ethics.py	<-- YOU
4. The reply (JSON) becomes a table	src/api_client.py	
5. Documents are found and downloaded	src/crawler.py	
6. Their text is read	src/pdf_extractor.py	<-- YOU
7. Scanned pages are read by OCR	src/ocr_extractor.py	
8. The table is tidied and checked	src/cleaner.py	
9. Charts are drawn	src/analysis.py	
10. Files are saved (CSV, Excel, JSON)	src/storage.py	

Two more pieces sit beside this path rather than on it: `src/static_scraper.py` shows the ordinary requests-and-BeautifulSoup method, and `src/selenium_scraper.py` shows the real-browser method. The Scrapy spider in `scrapy_project/` does the document search a second way, with the Scrapy framework.

Your part sits at the front and the middle of the project. At the front, `ethics.py` is step 3 - every request made by anybody's code goes through it. In the middle, `pdf_extractor.py` is step 6 - it reads the documents Sasini's crawler downloads, and tells Salaama's OCR which pages need reading as pictures.

Showing your part in the viva

The viva is 8 minutes for the whole team: about 3 minutes of presentation, then 5 minutes of questions about what each member did. This chapter gets you ready for your share of it.

Before the viva

1. Double-click `Run_CSE_App.bat` in the project folder. The app opens in your browser. Keep the black window open - closing it stops the app.
2. Check your tab works, as below.
3. Run your files once from the terminal, so you have seen the output yourself.

Showing it in the app

1. Open the **Ethics** tab. Your name is on the label at the top.
2. Press **Read robots.txt**. Point to the file on the left and the settings on the right - the one blocked section, the 1.5 second delay, and our User-Agent.
3. Press **Show the request log**. Show that every request is recorded with the time and the delay.
4. Open the **Announcements** tab and scroll to **Read the text out of a document**. Choose a typed document and press **Read the text** - show the page table and the file details.
5. Choose the de-listing document and read it again - show that it gives no text and says to use OCR.

Showing it from the terminal

```
python -m src.ethics          # robots.txt, a real request
python -m src.static_scraper  # 24 characters, then the sitemap
python -m src.pdf_extractor   # which PDFs need OCR
```

Your 30-second explanation

Practise saying this out loud until it feels natural. Change the words so they sound like you.

"My part is the rules, and reading the documents. Every request our project makes goes through one function I wrote, `fetch()`. Before anything is sent, it checks CSE's `robots.txt`, waits one and a half seconds, and says who we are in the User-Agent, and afterwards it writes the request to a log. I also wrote our first scraping attempt with requests and BeautifulSoup - it found only 24 characters, because the CSE page is built by JavaScript, and that is why we read the data addresses instead. Finally, I read the text out of the announcement PDFs with pdfplumber and PyPDF2, and spot the scanned ones that need OCR."

If the examiner points at your code

These are the parts of your files most likely to be pointed at, and what to say about each:

If they point at...	Say...
<code>allowed, reason = self.can_fetch(url)</code> at the top of <code>fetch()</code>	This is the permission check. It is the first line, so no request can skip it.

If they point at...	Say...
<code>wait()</code>	It measures the time since the last request and sleeps only for what is left of the 1.5 seconds.
<code>self.robots = None</code> in <code>read_robots()</code>	If <code>robots.txt</code> cannot be read, every request is refused. We do not guess in our own favour.
The <code>/cgi-bin/ ...</code> (allowed) output line	Python's parser uses the first matching rule, and CSE's empty Disallow line comes first. We never request <code>/cgi-bin/</code> anyway.
<code>want="text"</code> in <code>scrape_page()</code>	The page request goes through <code>fetch()</code> like every other, and comes back as text for BeautifulSoup.
<code>soup.get_text(strip=True)</code>	Every word on the page, with spaces trimmed - only 24 characters on CSE.
<code>page.extract_text()</code> or <code>""</code>	<code>extract_text</code> gives <code>None</code> on an empty page; this turns it into an empty string.
<code>config.OCR_TEXT_THRESHOLD</code> (50)	Under 50 characters means a scan. Not 0, because scans can carry a few stray characters.

Practice questions

These are the same 50 questions as your section of the team guide: about 30 on your own part, then 20 on the project as a whole. Cover each answer with your hand and try to say it first. By now, every answer should make sense - if one does not, go back to the chapter it comes from.

EASY - THE BASICS

1. Which files did you write?

`src/ethics.py`, which handles every request the project makes; `src/static_scraper.py`, the requests and BeautifulSoup part; and `src/pdf_extractor.py`, which reads the text out of the PDF documents.

2. In one sentence, what does `src/ethics.py` do?

It sends every web request the project makes, and applies the ethical rules to each one - checking `robots.txt`, waiting between requests, saying who we are, and writing everything to a log.

3. Where does the project read `robots.txt` from?

`https://www.cse.lk/robots.txt`. Every website keeps it at the top level of the site, at that exact name.

4. How often do you read `robots.txt`?

Once at the start of every run, in `read_robots()`. We do not keep a permanent copy, because a site can change its rules and an old copy would mean we were following permissions that had expired.

5. What is a User-Agent?

A short piece of text sent with every request that says what program is asking. Browsers send one that names the browser. Ours names the university, the course, the group and a contact email.

6. What is BeautifulSoup?

A Python library for pulling data out of HTML. You give it the HTML text, it works out the structure, and then you can ask it for tags, text or attributes by name.

7. Which parser did you use with BeautifulSoup?

`html.parser`, which comes with Python so nothing extra has to be installed. For the sitemap we use `xml` mode, which needs `lxml`, and the code falls back to `html.parser` if `lxml` is missing.

8. What does `find()` do, and how is `find_all()` different?

`find()` returns the first tag that matches. `find_all()` returns a list of every tag that matches.

9. Which two libraries do you use to read PDFs?

`PyPDF2` and `pdfplumber`. We use both, because they are good at different things.

10. What are the three types of PDF?

Text based, made by a computer, where the letters are real text. Image based, which is a scan - a picture of the words with no text inside. And hybrid, which has some of each, such as a report with digital tables and a scanned diagram.

11. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

12. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

13. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

14. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

15. What does CSE's robots.txt say?

It blocks one section, `/cgi-bin/`, and allows everything else. It does not set a `Crawl-delay`, so it does not ask us to wait between requests at all.

16. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

17. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

18. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

19. Walk me through what happens when `fetch()` is called.

Four steps, in this order.

First it asks `can_fetch()` whether `robots.txt` allows that address. If not, it stops there and records the attempt as skipped. Second it calls `wait()`, which sleeps until 1.5 seconds have passed since the last request. Third it sends the request through the session, which carries our User-Agent. Fourth it writes a row to `logs/request_log.csv` and checks the status code before returning the data.

```
def fetch(self, url, method="GET", data=None, save_as=None, want="json"):
    allowed, reason = self.can_fetch(url)      # 1. permission
    if not allowed:
        return {"ok": False, ...}
    self.wait()                               # 2. politeness
    response = self.session.post(url, ...)     # 3. the request
    self.log.log_request(method, url, ...)     # 4. the record
```

20. How does `can_fetch()` actually decide?

It uses `RobotFileParser`, which is part of Python's standard library. We give it the text of `robots.txt` with `parse()`, then call `can_fetch(user_agent, url)` and it returns `True` or `False`. We did not write the rule matching ourselves - the standard library already knows the format.

21. Why does `wait()` measure the time instead of always sleeping 1.5 seconds?

Because part of the delay may have passed already. If parsing the last reply took a second, we only need to wait another half second - the server still got its 1.5 seconds of breathing room either way. Sleeping the full amount every time would just make the program slower for no extra politeness.

```
def wait(self):
    since_last = time.time() - self.last_request_time
    if since_last < self.delay:
        time.sleep(self.delay - since_last)
    self.last_request_time = time.time()
```

22. Why does `fetch()` return a dictionary instead of raising an error?

Because it runs behind a web page. If an error were raised and not caught, it would stop the whole app and the screen would go blank. Returning `{"ok": False, "error": ...}` means the tab can show a readable message and everything else keeps working.

23. What is a `requests.Session` and why use one?

A Session keeps the connection to the server open between requests, instead of opening a new one each time. It saves the server a little work on every request, and it lets us set the User-Agent once instead of repeating it on every call.

24. What is in `logs/request_log.csv`?

One row for every request the project has ever sent: the time, whether it was GET or POST, the address, the status code that came back, whether robots.txt allowed it, and the delay we waited. It is the evidence for how the data was collected.

25. What happens if robots.txt cannot be read at all?

We treat every address as blocked. `can_fetch()` returns False with the reason that robots.txt could not be read. If we cannot see the rules we do not guess in our own favour - we stop.

26. Go through the BeautifulSoup methods you used.

`find()` for the first matching tag and `find_all()` for all of them. `select()` and `select_one()` do the same thing but with CSS selectors, which is handy for something like `div.price > span`. `get_text()` gives the words inside a tag, and `strip=True` removes the surrounding whitespace. `get()` reads one attribute, such as the href on a link.

```
soup.find("title")           # first matching tag
soup.find_all("a")           # every matching tag
soup.select_one("div.price")  # CSS selector
tag.get_text(strip=True)     # the words inside
tag.get("href")              # one attribute
```

27. What exactly did `requests` return for the CSE page?

About 25,000 bytes of HTML, with a status code of 200 - so the request itself worked perfectly. But asking BeautifulSoup for the text gave about 24 characters, and asking for tables gave none.

28. How do you know the problem was the page and not your code?

We pointed the same library, with the same methods, at CSE's sitemap - which is an ordinary file with no JavaScript - and it read all 15 pages listed without any trouble. Same tool, working result. That isolates the problem to the page.

29. How would you tell whether a page is static or dynamic before writing any code?

Right-click and choose View Page Source, which shows what the server actually sent. Then right-click and choose Inspect, which shows the page as it is now, after JavaScript has run.

If the numbers appear in Inspect but not in View Source, the page is built by JavaScript and requests will not see them.

30. What is the difference between PyPDF2 and pdfplumber?

PyPDF2 is the simpler one. It extracts text and gives us the file details - the page count, the author, and what the file was created with. It does not keep the layout well and it cannot find tables.

pdfplumber keeps the layout much better and can pull tables out using the position of the text on the page. That is why it is our main method, and we use PyPDF2 alongside it for the file details.

```
# PyPDF2 - simple text and the file details
reader = PdfReader(path)
text = reader.pages[0].extract_text()
details = reader.metadata

# pdfplumber - layout and tables
with pdfplumber.open(path) as pdf:
    text = pdf.pages[0].extract_text()
    tables = pdf.pages[0].extract_tables()
```

31. How does your code decide a PDF needs OCR?

It counts the characters on each page. If a page gives fewer than 50, we treat it as a scan and mark it for OCR.

We use 50 rather than zero because a scan sometimes carries a few stray characters that are not the real document text - a header added afterwards, for instance. Requiring a real amount of text means a handful of stray letters cannot fool us into skipping OCR.

32. Why do you write `page.extract_text() or ""` instead of just `page.extract_text()`?

Because `extract_text()` returns `None`, not an empty string, when there is nothing on the page. Calling `.strip()` or `len()` on `None` would crash. The `or ""` turns it into an empty string so the rest of the code can treat every page the same way.

33. Why did `requests` and `BeautifulSoup` not work on the CSE site?

Because the page is dynamic. `requests` downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

34. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

35. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

36. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

37. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

38. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

39. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

40. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

41. Somebody says the delay makes the project slow. Defend it.

The slowness is the ethics working, not a fault.

Collecting 15 company profiles takes about 25 seconds, and it could take under a second if we fired them all at once. Choosing not to is the whole point - it is the difference between behaving like a considerate visitor and behaving like a denial of service.

It also costs us almost nothing in practice, because the main dataset arrives in a single request. The delay only shows when we deliberately loop over many companies.

42. What did you deliberately choose not to do, and why?

We did not use any technique for getting around a security control. No logins, no paywalls, and no CAPTCHA solving.

We also did not rotate User-Agents or hide who we are. Plenty of scraping tutorials tell you to copy a browser's User-Agent so the site cannot tell you apart from a person. We do the opposite and name ourselves, because the point of the header is to identify the traffic, not to disguise it.

43. How would you prove to the lecturer that you did not overload their server?

By opening `logs/request_log.csv`. It has one row per request with a timestamp, so you can see the requests are spaced at least 1.5 seconds apart, and count them - roughly 30 to 40 for a full run.

The Ethics tab in the app shows the same file on screen.

44. Why did you keep code in the project that does not get the data?

Because it explains the design. Without it, somebody reading the project would reasonably ask why we are not just using BeautifulSoup like everybody else.

`src/static_scraper.py` answers that in one run: it shows the ordinary approach returning almost nothing on this particular site, and then shows the same library working fine on a normal file. It is the reasoning behind the whole project, written as code you can execute.

45. If CSE were a normal static site, what would change?

We would use `src/static_scraper.py` as the main collector instead of the API client. We would find the table in the HTML, loop over the `<tr>` rows, take the `<td>` cells with `get_text(strip=True)`, and build the DataFrame from those.

The ethics layer, the cleaning, the PDF work and the charts would all stay exactly the same, because they do not care where the rows came from.

46. What makes extracting text from PDFs difficult in general?

A PDF is a set of drawing instructions, not a document with a structure. It says where to put each character, not which paragraph it belongs to. So the library has to guess at things a human reads instantly.

Tables are the worst case - they can span pages, have merged cells, and there is nothing in the file saying 'this is a table'. Line breaks are ambiguous, because a break at the edge of the page might be a new paragraph or just the line running out. Headers and footers repeat on every page and you may or may not want them. And formatting like bold, italics and ligatures often comes out wrong.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

Word list

Every special word used in this document, from A to Z.

Word	What it means
Attribute	Extra information inside an HTML tag, like href in a link.
BeautifulSoup	A Python library that reads HTML and lets you find parts of it.
CAPTCHA	A test a website uses to check a visitor is a person.
Crawl-delay	A robots.txt line asking programs to wait between requests. CSE does not have one.
CSV	A simple table file: one row per line, commas between columns.
DataFrame	A table in pandas, with rows and named columns.
Disallow	A robots.txt line naming a part of the site programs should not visit.
Dynamic page	A page whose content is added by JavaScript later.
Function	A named set of steps that can be called to do a job.
HTML	The language web pages are written in, using tags.
Hybrid PDF	A PDF with some typed pages and some scanned pages.
JavaScript	A language that runs inside the browser and can change the page after it loads.
JSON	A tidy text format for data: names and values.
Library	Code somebody else wrote that we can import and use.
Metadata	Facts stored inside a file about the file itself, such as its author.
OCR	Optical Character Recognition - reading letters from a picture.
Parser	The part of a library that reads text and works out its structure.
PDF	A document file that looks the same on every computer.
pdfplumber	A PDF library that keeps the layout and can find tables.
PyPDF2	A simpler PDF library that also reads the file details.
Rate limiting	Limiting how many requests are sent in a given time.
Request	A message asking a server for something.
Response	The server's reply to a request.
RobotFileParser	The part of Python that reads robots.txt and answers 'may I fetch this?'
robots.txt	A file where a website says which parts programs may visit.
Scanned PDF	A PDF that holds a photograph of a page, with no real text inside.
Server	A computer that stores a website and answers requests.
Session	A connection to a server kept open for several requests.
Share	A small piece of a company that people can buy and sell.

Word	What it means
Sitemap	A file where a website lists its own pages for programs.
Static page	A page whose content is already in the HTML file.
Status code	A number on each reply saying how it went - 200 means OK.
Tag	A label in HTML, such as <title> or <table>.
Text-based PDF	A PDF made by a computer, with real letters inside.
Tuple	A fixed group of values in round brackets, like (name, explanation).
URL	The address of a page or file on the web.
User-Agent	The part of a request that says which program sent it.
Variable	A name that holds a value.
XML	A text format like HTML, where the website chooses its own tag names.